Guía práctica: Desarrollo de software con Claude Code

Bloques 3 y 4: Front end, mantenimiento y entrega

Ingeniería de Software · Universidad Iberoamericana León

21 de octubre de 2026 · M.I. Ricardo Medrano

1. Introducción

Al terminar los bloques 3 y 4 tu sistema estará completo y entregado: las pantallas de tu prototipo funcionando contra la API real, al menos un error corregido y un cambio de requerimiento implementado, y una **versión 1.0** que cualquiera puede levantar desde cero siguiendo solo tu README.

Esta guía continúa la de los bloques 1 y 2. Supone que ya tienes `CLAUDE.md`, los tres contenedores, la API con tus reglas de negocio y el servidor MCP de solo lectura.

Objetivos de aprendizaje

1. Convertir un prototipo en un front end conectado a una API, sin ampliar el alcance.
2. Verificar el trabajo del agente contra el sistema real, no contra simulaciones.
3. Corregir errores con el flujo profesional: reproducir, prueba que falla, corregir, prueba que pasa.
4. Implementar cambios de requerimiento con trazabilidad: solicitud, base de datos, API, front, pruebas y documentación.
5. Revisar código y decidir qué se corrige y qué no, con justificación.
6. Demostrar que el proyecto es entregable: se levanta desde cero en otra carpeta solo con el README.

Mapa de los bloques 3 y 4

Bloque	Pasos en clase	Pasos de tarea	Resultado
3. Front end desde el prototipo	3.0 a 3.5	—	Pantallas conectadas a la API y verificadas en el navegador
4. Mantenimiento y entrega	4.1, 4.2, 4.6, 4.7, 4.8	4.3, 4.4, 4.5	Error corregido, cambio implementado, revisión de código, README y versión 1.0

> Regla de oro, segunda parte: **“funciona en mi máquina” no es lo mismo que “se puede entregar”.** El último paso de esta guía lo demuestra.

2. Bloque 3: Front end desde el prototipo

Al terminar este bloque las pantallas de tu prototipo trabajarán con la API real en lugar de datos en memoria, y Claude las habrá verificado en el navegador. Duración: 2 horas.

Paso 3.0: Prepara la sesión y una regla nueva

Abre una **sesión nueva** en la carpeta de tu proyecto. Antes de construir, agrega una regla a tu memoria del proyecto (modo **Accept edits**):

Prompt:

```
Agrega a las reglas de trabajo de CLAUDE.md: "Verificar siempre contra el
stack real con docker compose. Si Docker no está disponible, detente y
avísame; no uses servidores simulados ni des una tarea por terminada sin
esa verificación."
```

Qué observar: en la prueba piloto, con Docker apagado, Claude probó el front contra un servidor simulado que él mismo inventó y dio la tarea por terminada. Esta regla lo evita. `CLAUDE.md` crece con cada incidente.

Antes de seguir, abre Docker Desktop y levanta el stack con `docker compose up -d`.

Paso 3.1: Planea la conexión con restricción de alcance

Cambia el modo a **Plan**.

Prompt:

```
Planea la conexión del front con la API:
- Reemplaza la capa de datos en memoria de web/index.html por llamadas
  fetch a /api/ (mismo origen a través de nginx).
- Todas las pantallas del prototipo deben usar los endpoints reales.
- Conserva el diseño y los estados vacío, de carga y de error del prototipo.
  Los errores de negocio (409, 404, 422) se muestran al usuario con el
  mensaje en español que devuelve la API.
- Separa el JavaScript en archivos dentro de web/js/ (cliente de la API y
  lógica de pantallas).
No cambies la API; si falta algún endpoint o campo, pregúntame antes.
```

Qué observar: la última línea es una **restricción de alcance**. En el piloto, el prototipo tenía un botón "Deshacer devolución" que la API no soportaba; Claude se detuvo y preguntó en lugar de agregar el endpoint por su cuenta.

Qué decidir: si tu prototipo tiene una función que la API no soporta, lo normal es **quitarla del front ahora** y registrarla como solicitud de cambio para el bloque 4. No amplíes el alcance a mitad del

bloque.

Verificación: revisa el plan antes de aprobarlo.

1. ¿Escapa el texto antes de insertarlo en el HTML (protección contra XSS)?
2. ¿Agrega estados de carga y de error de red que el prototipo no tenía?
3. ¿Refresca los datos después de cada alta, devolución o error 409?
4. ¿Menciona riesgos? En el piloto detectó uno real: la fecha "hoy" del navegador y la del servidor difieren después de las 18:00 (el contenedor corre en UTC).

Si Claude detecta un riesgo que no corresponde a este bloque, pídele que lo registre:

```
No lo corrijas ahora; regístralo en docs/pendientes.md con su descripción
y cómo reproducirlo, para atenderlo después como mantenimiento.
```

Paso 3.2: Implementa y lee el reporte con lupa

Aprueba el plan y cambia a **Accept edits**.

Verificación: al terminar, Claude entrega un reporte. Busca tres cosas:

- **Qué no pudo verificar.** Si dice que algo no lo vio funcionar, no está terminado.
- **Diferencias con el plan.** Si se desvió, ¿te lo justificó? ¿Debió preguntar antes?
- **Defectos que reconoce y deja pasar.** En el piloto, un botón de filtro quedaba seleccionado aunque la lista no se actualizara. Pide que lo corrija.

Paso 3.3: Claude prueba su trabajo en el navegador

Abre el panel **Browser** (`Ctrl + Shift + B`, o `Cmd + Shift + B` en Mac) para verlo en vivo.

Prompt (ejemplo de LabPresta; adapta el flujo al de tu sistema):

```
Abre localhost:8080 en el navegador y recorre el flujo completo contra la
API real:
1. Filtra el catálogo por estado "disponible".
2. Registra un préstamo de un equipo disponible.
3. Intenta prestar ese mismo equipo otra vez; debe mostrarse el error 409.
4. En préstamos activos, registra la devolución.
5. Confirma que el equipo volvió a "disponible" en el catálogo.
6. Ejecuta docker compose stop api, recarga la página y confirma que aparece
   el estado de error con el botón Reintentar. Luego docker compose start api
   y usa Reintentar.
Toma capturas en cada paso y repórtame lo que no funcione.
```

Qué observar: el punto 6 prueba un estado de error que Claude implementó pero no había visto funcionar. Los casos de falla también se prueban.

Paso 3.4: Comparación visual contra el prototipo

Prompt:

```
Abre el prototipo en la carpeta prototipo/ y localhost:8080, compara las
pantallas y lista las diferencias visuales. No corrijas nada todavía.
```

Qué decidir: algunas diferencias son correctas (datos reales en lugar de los de ejemplo, un botón retirado por alcance). Corrige solo las que sean errores.

Paso 3.5: Revisión humana y commit

Abre **Changes** y deja al menos un **comentario sobre una línea** pidiendo una mejora; envíalo con `Ctrl + Enter`. Después haz tú mismo el recorrido en tu navegador normal y pide el commit.

Resultado esperado del bloque 3

```
web/
├── index.html      (solo HTML y CSS; carga los scripts)
├── nginx.conf
└── js/
    ├── util.js     (utilidades, escape de texto)
    ├── api.js      (cliente de la API y manejo de errores)
    └── app.js      (pantallas, estado y eventos)
docs/pendientes.md  (riesgos detectados, con cómo reproducirlos)
```

3. Bloque 4: Mantenimiento y entrega

El sistema ya funciona; ahora hay que **cambiarlo sin romperlo** y demostrar que se puede entregar. Duración: 2 horas en clase para los pasos obligatorios (4.1, 4.2, 4.6, 4.7 y 4.8); los pasos 4.3, 4.4 y 4.5 son de tarea.

Paso 4.0: Sesión nueva

Abre una **sesión nueva** en modo **Plan**, con el stack levantado. Si aún no existe, Claude creará `docs/solicitudes_cambio.md` en el paso 4.2.

Paso 4.1: Corrige un error real (obligatorio)

Toma un pendiente de tu `docs/pendientes.md`. Si no tienes ninguno, usa el de LabPresta: la fecha "hoy" del contenedor (UTC) difiere de la de México después de las 18:00.

Prompt:

```
Atiende el pendiente <nombre> de docs/pendientes.md siguiendo este orden:
1. Escribe primero una prueba con pytest que reproduzca el error y confirma
   que falla.
2. Propón la corrección; explícame las opciones antes de elegir una.
3. Aplícala y confirma que la prueba ahora pasa, junto con las existentes.
4. Marca el pendiente como resuelto, con la fecha y la prueba de regresión.
```

Qué observar: el flujo profesional es **reproducir, prueba que falla, corregir, prueba que pasa**. Si Claude corrige sin reproducir primero, no hay evidencia de que lo arregló.

Qué decidir: en LabPresta las opciones eran fijar `TZ` en el contenedor, calcular "hoy" con una zona horaria explícita en el código, o que el front use la fecha del servidor. Se eligió la zona explícita y configurable (`api/app/fechas.py`). Justifica tu elección en la bitácora.

Verificación: `docker compose exec api pytest` en la Terminal; todas las pruebas pasan.

Paso 4.2: Cambio de requerimiento con migración (obligatorio)

Prompt (ejemplo de LabPresta; usa un cambio que agregue un campo a tu modelo):

```
Llegó una solicitud de cambio del cliente: "Al registrar una devolución,
el personal necesita anotar observaciones opcionales sobre el estado del
equipo (por ejemplo: 'cable dañado')".
1. Regístrala en docs/solicitudes_cambio.md con folio SC-001, descripción,
   impacto (BD, API, front, pruebas) y estado.
2. Planea la implementación: nueva columna con migración de Alembic, cambio
   en el esquema y en el endpoint, campo en la pantalla y una prueba.
```

Qué observar: un cambio "pequeño" toca cuatro capas. En el piloto, Claude hizo un commit por capa: registro, base de datos, API, pruebas, front y documentación. El historial de Git cuenta la historia del cambio.

Verificación:

1. `\d <tabla>` con psql muestra la columna nueva; los registros anteriores siguen ahí.
2. Por MCP: *"¿Qué columnas tiene ahora la tabla <tabla>?"*
3. En pantalla, el campo nuevo se guarda y se consulta.
4. `CLAUDE.md` menciona la solicitud de cambio implementada.

Paso 4.3: Nueva regla de negocio (tarea)

Prompt (ejemplo de LabPresta):

```
Solicitud de cambio SC-002: "Un solicitante no puede tener más de 2
préstamos activos al mismo tiempo". Regístrala en docs/solicitudes_cambio.md
como RN4 y actualiza la sección de reglas de negocio del plan en docs/.
Implementa primero las pruebas (que deben fallar), luego la regla en la API
con un 409 y mensaje en español, y verifica que el front muestre el mensaje.
```

Qué observar: el plan de proyecto también se mantiene. Si la regla vive en el código pero no en el plan, la documentación miente.

Paso 4.4: Diseño antes que código (tarea)

Prompt (en modo Plan):

```
Solicitud de cambio SC-003: "<una función que tu prototipo tenía y retiraste
en el bloque 3, o una que pida tu cliente>". No implementes nada. Analiza el
impacto y hazme las preguntas de diseño que necesites resolver antes de
poder implementarlo.
```

Qué observar: lo más valioso del agente aquí son **las preguntas**. En LabPresta, "deshacer una devolución" llevó a ocho decisiones registradas en SC-003:

1. Solo el mismo día de la devolución.
2. Si el solicitante queda con 3 préstamos, se permite como excepción a RN4.
3. Se deshace desde una sección de devoluciones recientes.
4. Se borran las observaciones y la fecha de devolución.
5. Queda registro de quién y cuándo.

6. Cualquiera puede deshacer (no hay autenticación en la Fase 1).
7. Una sola vez por préstamo.
8. No se deshace si el equipo pasó a mantenimiento.

Responde tú las preguntas como si fueras el cliente y documenta cada decisión. Implementarlo es opcional.

Paso 4.5: Deshacer cambios (tarea)

Prompt:

```
Cambia el color principal del sistema a rojo en todo el front.
No hagas commit.
```

Revísalo en el navegador y luego reviértelo por alguno de estos caminos:

Situación	Cómo regresar
El cambio no tiene commit	Escribe `/rewind` y elige el punto anterior, o en la Terminal `git restore .`
El cambio ya tiene commit	`git revert <commit>`: crea un commit nuevo que lo deshace sin borrar historia

Qué observar: en el piloto, sin la frase "No hagas commit", Claude guardó el experimento en dos commits y hubo que revertir ambos. Para experimentar, díselo explícitamente o trabaja en una rama.

Paso 4.6: Revisión de código (obligatorio)

Escribe `/code-review`. La app genera una tarjeta de revisión con las opciones *Walk through in diff* y *Apply fixes*.

No apliques todo automáticamente. Registra cada hallazgo en `docs/revision_codigo.md`:

Hallazgo	Severidad	Decisión	Justificación
Ejemplo: falta validar longitud máxima en un campo	Media	Aceptado	Corregido en el commit abc123
Ejemplo: sugiere paginar el catálogo	Baja	Descartado	El MVP tiene menos de 50 equipos; queda como pendiente

Qué observar: decidir que algo no se corrige también es una decisión de ingeniería, siempre que quede justificada.

Paso 4.7: README basado en el código real (obligatorio)

Prompt:

```
Genera un README.md en español con: descripción del sistema, arquitectura
(los tres contenedores y el MCP), requisitos, cómo levantarlo desde cero en
Windows y en Mac (incluido copiar .env y cargar los datos de ejemplo), cómo
ejecutar las pruebas, endpoints principales, reglas de negocio vigentes,
solicitudes de cambio atendidas y problemas comunes. Básalo en el código
real, no en el plan.
```

Qué observar: si el README se escribe a partir del plan, describe lo que se pensaba hacer, no lo que existe.

Paso 4.8: Prueba de entrega desde cero (obligatorio)

Es la verificación más honesta del curso: ¿se puede levantar tu sistema siguiendo solo el README?

1. **Detén tu stack original:** en la carpeta del proyecto, `docker compose down`.
2. **Clona en otra carpeta**, fuera del proyecto:

   ```
   git clone <ruta-de-tu-proyecto> prueba-entrega
   cd prueba-entrega
   ```

3. **Copia el archivo de variables:** `cp .env.example .env` en Mac, o `copy .env.example .env` en PowerShell.
4. **Sigue solo el README** para levantar, cargar datos y abrir el sitio.

Si algo falla, diagnostica sin cambiar nada. Abre una sesión de Claude Code en `prueba-entrega`:

```
El sistema no funciona desde cero: <describe lo que ves>. Diagnostica con
docker compose ps -a y docker compose logs api, explícame la causa y propón
la corrección. No cambies nada todavía.
```

La corrección va **en el repositorio original**, nunca en el clon. Después borra la prueba (`docker compose down -v` dentro de `prueba-entrega`; aquí sí va `-v`, porque borra solo la base de prueba), elimina la carpeta y repite hasta que funcione a la primera.

Qué observar: en el piloto, el sistema "funcionaba" desde hacía días, pero la prueba desde cero encontró dos problemas: un estado inconsistente de Docker y una base sin datos, porque el README no incluía el paso de carga de datos de ejemplo.

Cierre de la versión 1.0

Prompt:

```
Antes de cerrar la versión 1.0, actualiza CLAUDE.md al estado real del
proyecto: lo que ya está implementado, las reglas de negocio vigentes y la
estructura de carpetas real. Haz commit.
```

Después, en la Terminal:

```
git tag v1.0
```

Qué observar: en el piloto, `CLAUDE.md` todavía decía que el front no usaba la API. Una memoria desactualizada engaña a las sesiones futuras: actualízala con cada cambio.

4. Control de cambios

Cada cambio de requerimiento sigue el mismo flujo, y cada etapa deja un commit. Si el historial de Git muestra este orden, el cambio es trazable.

1. **Registro**: la solicitud entra a `docs/solicitudes_cambio.md` con folio, descripción e impacto.
2. **Plan de proyecto**: si cambia una regla de negocio o el alcance, se actualiza el plan en `docs/`.
3. **Base de datos**: migración de Alembic, revisada antes de aplicarla.
4. **Pruebas**: se escriben antes de la implementación y deben fallar.
5. **API**: la implementación hace que las pruebas pasen.
6. **Front**: la pantalla refleja el cambio y muestra los errores nuevos.
7. **Documentación**: la solicitud pasa a *Implementada* y `CLAUDE.md` se actualiza.

Plantilla de solicitud de cambio

```
## SC-00X — <título corto>

- **Fecha de registro:** AAAA-MM-DD
- **Estado:** Registrada | En análisis | Implementada | Rechazada
- **Solicitud del cliente:** «<texto tal como lo pidió>»

**Descripción.** Qué cambia y qué regla de negocio afecta (nueva o modificada).

**Decisiones de diseño.** Las preguntas que se resolvieron antes de implementar.

**Impacto.**

| Capa | Cambio |
| --- | --- |
| BD | |
| API | |
| Front | |
| Pruebas | |

**Implementación.** Migración, archivos y pruebas; verificado con el stack real.
```

5. Solución de problemas de los bloques 3 y 4

El primer paso no cambia: **copia el error completo y pégaselo a Claude**, y pide un diagnóstico sin cambios (*"explícame la causa y propón la corrección; no cambies nada todavía"*). Estos casos aparecieron en la prueba piloto.

Síntoma	Causa	Solución
Docker no funciona después de instalarlo o actualizar WSL	Falta reiniciar el equipo	Reinicia la computadora y vuelve a abrir Docker Desktop
Claude dice que probó "con un servidor de prueba" o que no pudo verificar	Docker Desktop estaba apagado	Enciéndelo, `docker compose up -d` y pide repetir la verificación. Agrega la regla del paso 3.0 a `CLAUDE.md`
El sitio muestra "Cargando..." unos 20 segundos y luego un error 502	El contenedor `api` está detenido	`docker compose ps -a` y `docker compose logs api`; arráncalo con `docker compose start api`
El formulario rechaza la fecha de hoy después de las 18:00	El contenedor corre en UTC y su "hoy" ya es mañana	Usa una zona horaria explícita en la API (paso 4.1)
Un filtro queda seleccionado aunque la lista no cambió	El front no regresa el estado de la pantalla tras un error de red	Pide que el estado visual coincida siempre con los datos mostrados
La columna nueva no aparece en la base	La migración no se aplicó	`docker compose exec api alembic upgrade head` y revisa con `\d <tabla>`
El MCP deja de conectar después de cambiar `LECTOR_PASSWORD`	`.mcp.json` lee la contraseña de una variable de tu sistema, no de `.env`	Define la variable (`setx` en Windows, `export` en Mac) y reinicia la app
Un experimento quedó guardado en commits	Claude hace commit por defecto	`git revert <commit>`; la próxima vez pide *"No hagas commit"* o usa `/rewind` antes del commit
Prueba desde cero: la API termina con `Exited (1)` y `Name or service not known`	El contenedor `db` quedó sin red por un estado inconsistente de Docker	`docker compose down` y `docker compose up -d` (sin `-v`)
Prueba desde cero: el sitio carga pero sin datos	Los datos de ejemplo no se cargan solos	`docker compose exec api python -m app.seed` y agrégalo al README

6. Lista de verificación y entregables finales

Marca cada punto antes de dar por terminado un bloque. La entrega final es el repositorio con la etiqueta `v1.0`.

Bloque 3

- [ ] Regla de verificación contra el stack real en `CLAUDE.md`
- [ ] Todas las pantallas del prototipo usan la API real
- [ ] Los errores de negocio se muestran con el mensaje en español de la API
- [ ] Estados de carga y de error de red probados (con `docker compose stop api`)
- [ ] Recorrido completo verificado en el panel Browser, con capturas
- [ ] Funciones fuera de alcance retiradas y registradas como pendiente o solicitud de cambio
- [ ] Al menos un comentario de revisión en Changes

Bloque 4

- [ ] Un pendiente corregido con prueba de regresión (4.1)
- [ ] Una solicitud de cambio con migración implementada en todas las capas (4.2)
- [ ] `docs/solicitudes_cambio.md` con folio, impacto, decisiones y estado de cada solicitud
- [ ] `docs/revision_codigo.md` con cada hallazgo aceptado o descartado y su justificación (4.6)
- [ ] README basado en el código real, con instrucciones para Windows y Mac (4.7)
- [ ] Prueba desde cero exitosa en otra carpeta, siguiendo solo el README (4.8)
- [ ] `CLAUDE.md` actualizado al estado real
- [ ] `docker compose exec api pytest`: todas las pruebas pasan
- [ ] Etiqueta `v1.0` creada
- [ ] Tarea: pasos 4.3, 4.4 y 4.5

Estructura final del repositorio

```
mi-proyecto/
├── CLAUDE.md   README.md   docker-compose.yml   .mcp.json
├── .env.example   .gitignore   .gitattributes
├── api/        app/ (modelos, esquemas, routers), alembic/, tests/
├── web/        index.html, nginx.conf, js/
├── prototipo/  (referencia visual, sin cambios)
└── docs/
    ├── plan_proyecto        (actualizado con las reglas vigentes)
    ├── pendientes.md
    ├── solicitudes_cambio.md
    ├── revision_codigo.md
    └── bitacora_prompts.md
```

Bitácora de prompts

Continúa `docs/bitacora_prompts.md` con la misma estructura de los bloques 1 y 2. En estos bloques registra especialmente:

- Cada vez que Claude se detuvo a preguntar, y qué decidiste.
- Cada vez que su reporte dijo que algo no pudo verificarse, y cómo lo resolviste.
- Lo que falló en la prueba desde cero y dónde lo corregiste.